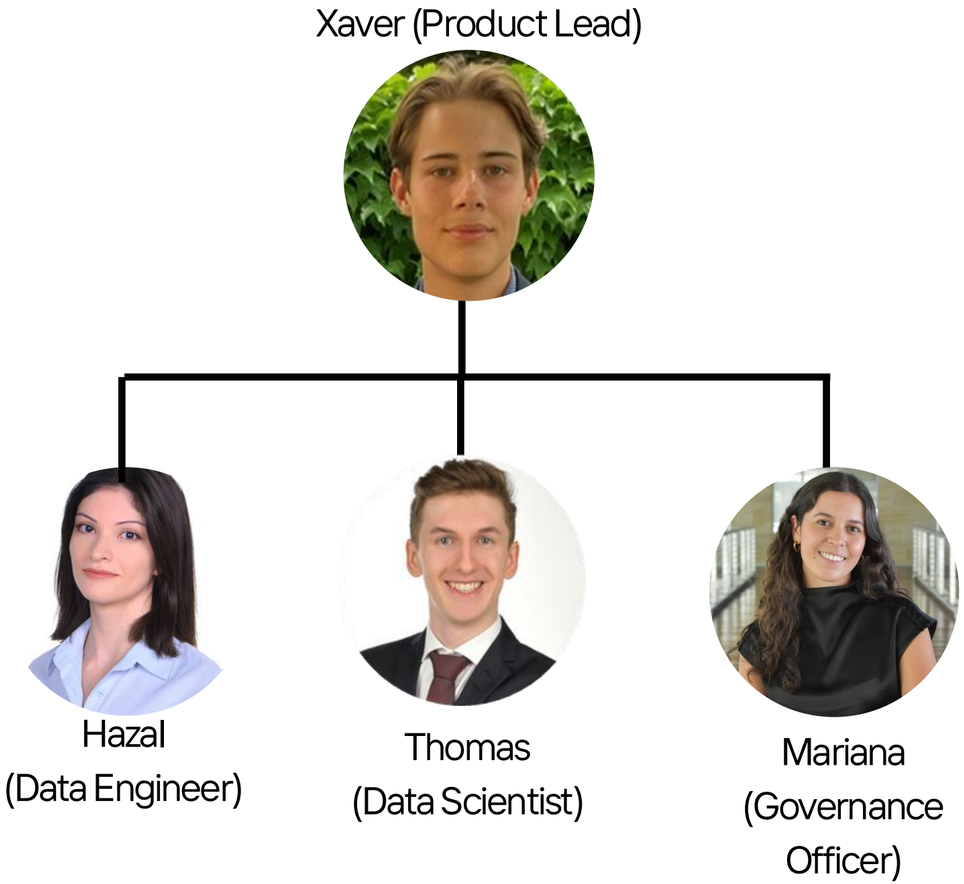
Data Ecosystem & Governance Project

Group 7B

Thomas Teichmann (74817)
Xaver Baeppler (72831)
Hazal Aksu (74918)
Mariana Marques Temudo (70262)

NOVA SCHOOL OF BUSINESS & ECONOMICS

Introduction & Dataset explained

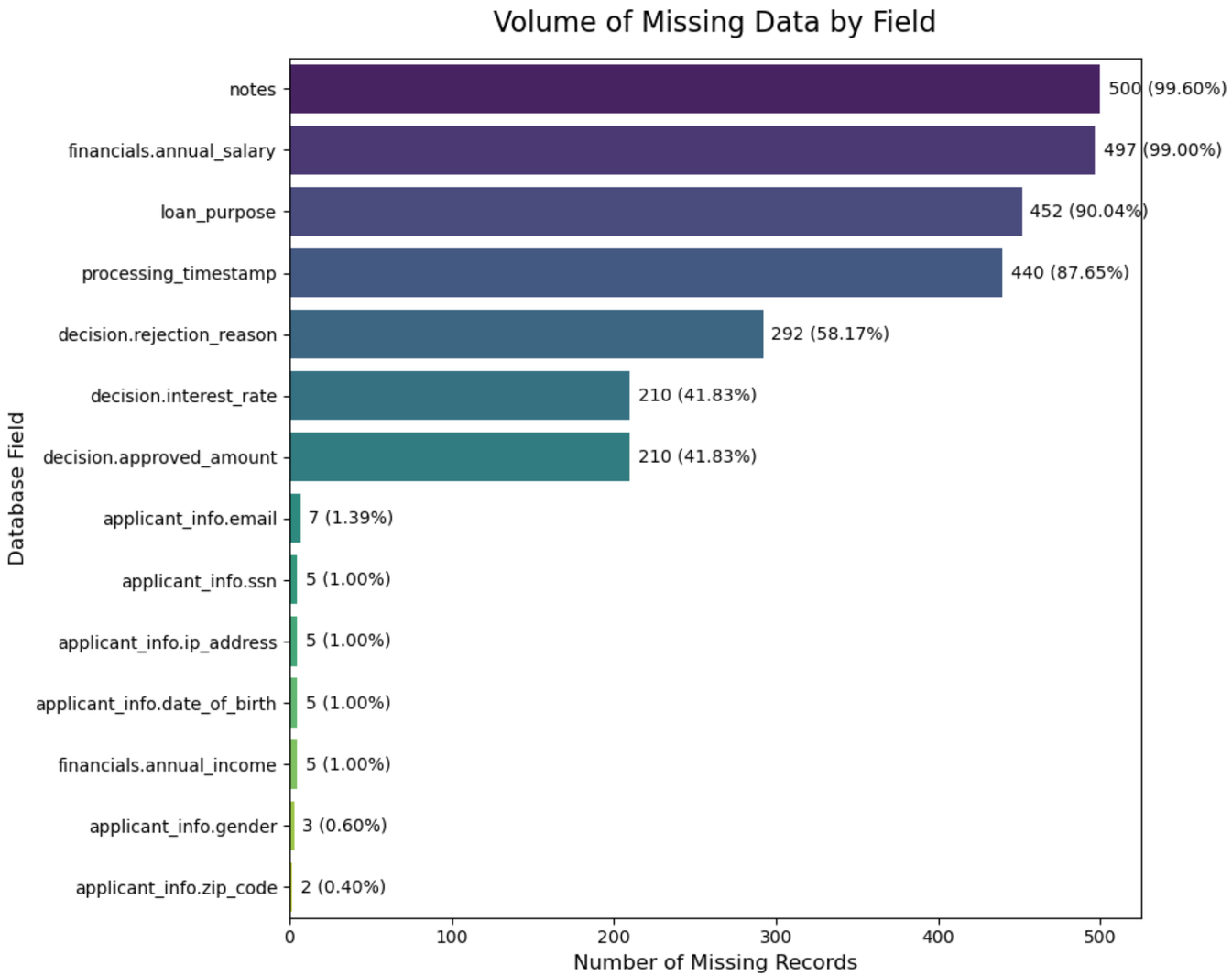


Data Engineering Findings

- Total records in raw data = 502
- *"annual_salary"* and *"annual_income"* show semantic heterogeneity
- *"annual_income"* contains 3 data types: integer, string and float
- Hidden fields: *"notes "* and *"annual_salary"*
- 1 duplicate entry in the dataset

```
Duplicates found: 1 (0.20%)
```

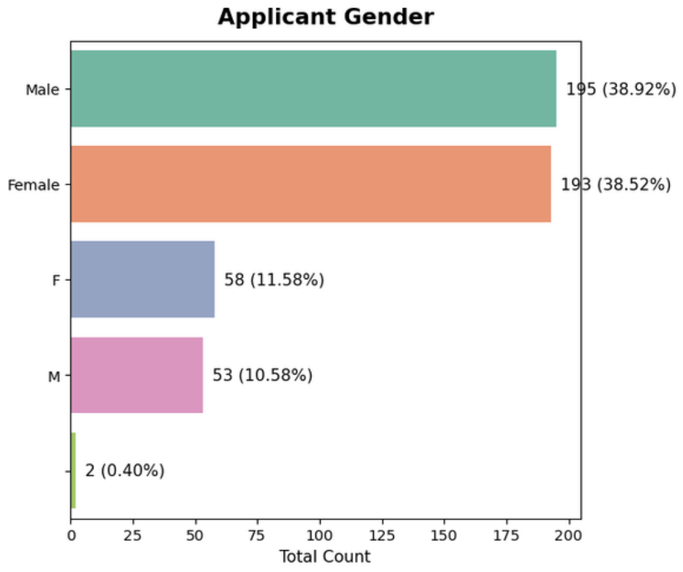
Data Engineering Findings

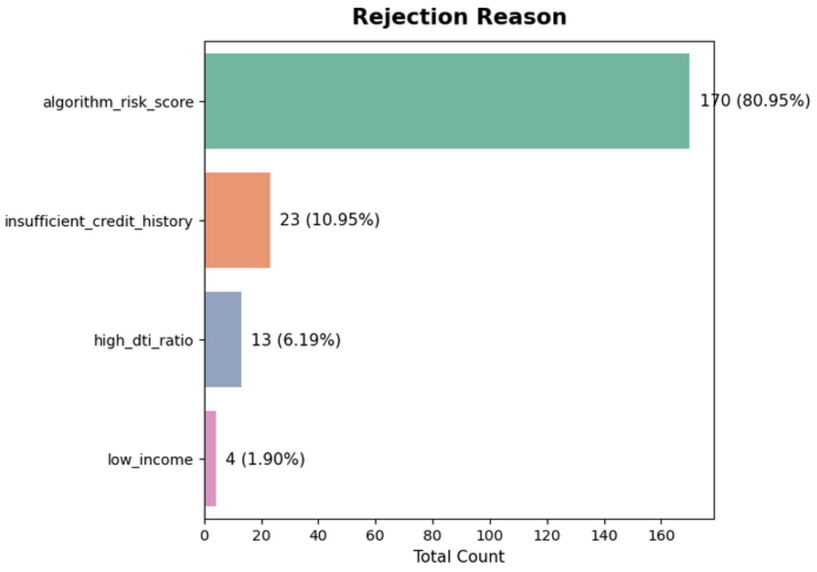


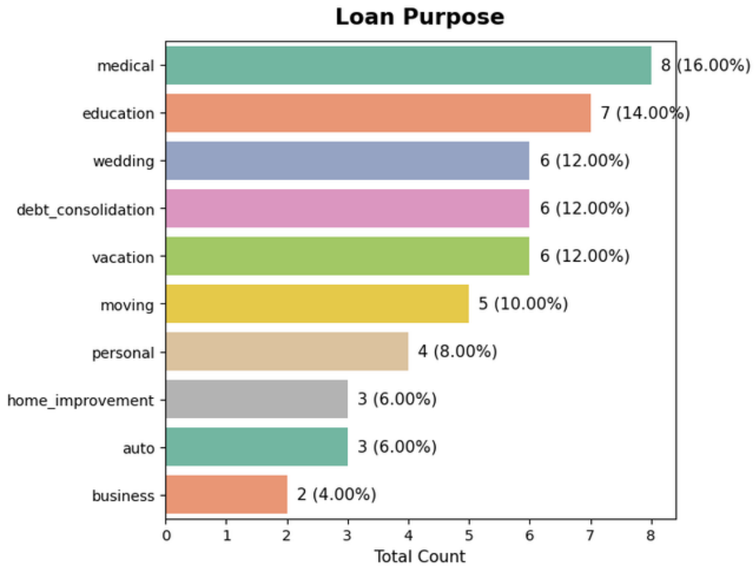
- The "*interest_rate*"and *"approved_amount"* sections are each missing 210 values while "*rejection_reason"* is missing 292 values. In total they make up 502, which is the number of entries in the data. This makes sense from the business perspective because approved loans don't have rejection reasons, and rejected loans don't have interest rates or approved amounts.

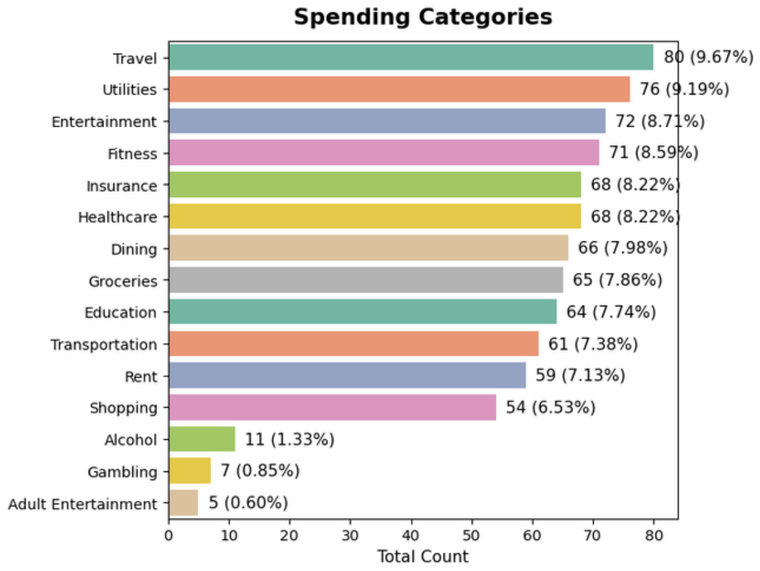
- The hidden "notes" section is nearly entirely composed of missing values at 500 (99.6%)

Data Engineering Findings









- The “gender” field contains five categories: “F”, “M”, “Female” , “Male” and “”

Data Engineering Findings

```
4. Inconsistent Data Types Analysis:
   - financials.annual_income: 8 records (1.59%) have incorrect type.
     -> Expected: 'number', Found: 'str' (Sample value: 55000)

6. Invalid & Impossible Values Analysis:
   - financials.credit_history_months: 2 records have impossible negative values.
   - financials.savings_balance: 1 records have impossible negative values.
   - applicant_info.email: 11 records have badly formatted emails.

7. SSN Formatting Analysis:
   - All existing SSNs are perfectly formatted as AAA-GG-SSSS.

8. Date Formatting Analysis:
   - date_of_birth: 161 records do not match 'YYYY-MM-DD'.
   - processing_timestamp: All existing records match ISO 8601 format.
```

Data Curation Process

Data Quality Dimension	Problem	Case	Remedy
Consistency	Duplicates in data	1 duplicate entry found	Delete the duplicates
Completeness	Missing Values	Entries with missing values found	If deemed unnecessary or detrimental to analysis, delete the rows that contain missing values from the dataset
Validity	Invalid/Impossible Values	11 badly formatted "email" values, 2 negative "credit_history_months" values and 1 negative "savings_balance" value found	Delete entries that contain invalid/impossible values
Consistency	Semantic Heterogeneity	"annual_income" and "annual_salary" are the same concept	Map annual_salary to annual_income and drop annual_salary from the dataset
Consistency	Non-standard Date Formats	"date_of_birth" not in a standard format across the entire dataset	Standardize all "date_of_birth" values into "YYYY-MM-DD" format

Data Curation Process

Data Quality Dimension	Problem	Case	Remedy
Completeness	Missing Keys	Some entries missing keys	If the key will not be entirely deleted from the dataset, inject placeholder values instead
Consistency	Non-standard Data Types	“annual_income” containing string, integer and float values	Convert and standardize all “annual_income” values to float type
Consistency	Non-standard Category Encoding	“gender” containing values “F”, “M”, “Female”, “Male”, “”	Map “F” and “M” values to “Female” and “Male” respectively. Delete entries with the blank “” value as it is detrimental to analysis

Data Curation Code

3.1 Delete Duplicates

```python
# Prepare the deletion operations
bulk_operations = []

for duplicate_group in duplicates:
    # duplicate_ids contains all identical records.
    # We slice the list [1:] to keep the first record and delete the rest.
    ids_to_delete = duplicate_group['duplicate_ids'][1:]

    for mongo_id in ids_to_delete:
        bulk_operations.append(DeleteOne({"_id": mongo_id}))

# Execute the deletions in a single batch
if bulk_operations:
    result = collection.bulk_write(bulk_operations)
    print(f"Success! duplicates resolved.")
    print(f" -> Documents permanently deleted: {result.deleted_count}")
else:
    print("No strict duplicates found to remediate.")
```

```
Success! duplicates resolved.
 -> Documents permanently deleted: 1
```

- The 1 duplicate entry was deleted
- Documents that contained negative *"credit_history_months"* and *"savings_balance"* values were deleted
- Documents that contained badly formatted values in *"email"* were deleted

3.2 Delete Entries with Invalid or Impossible Values

```python
print("\n--- DELETING RECORDS WITH IMPOSSIBLE VALUES & BAD EMAILS ---")

# 1. Dictionary of our negative value queries
negative_queries = {
    "financials.credit_history_months": {"$lt": 0},
    "financials.savings_balance": {"$lt": 0},
    "financials.annual_income": {"$lt": 0},
    "financials.annual_salary": {"$lt": 0},
    "decision.approved_amount": {"$lt": 0},
    "spending_behavior.amount": {"$lt": 0}
}

total_deleted_financials = 0

# Loop through and delete
for field, query_condition in negative_queries.items():
    query = {field: query_condition}
    result = collection.delete_many(query)

    if result.deleted_count > 0:
        print(f" -> Deleted {result.deleted_count} records due to negative '{field}'.")
        total_deleted_financials += result.deleted_count

# 2. Delete badly formatted emails
email_regex = r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"
invalid_email_query = {
    "applicant_info.email": {
        "$exists": True,
        "$type": "string", # Ensures we only check string types
        "$not": {"$regex": email_regex}
    }
}

email_result = collection.delete_many(invalid_email_query)
if email_result.deleted_count > 0:
    print(f" -> Deleted {email_result.deleted_count} records due to badly formatted emails.")

# Summary
total_purged = total_deleted_financials + email_result.deleted_count
print(f"\nTotal invalid records successfully purged from database: {total_purged}")
```

```
--- DELETING RECORDS WITH IMPOSSIBLE VALUES & BAD EMAILS ---
 -> Deleted 2 records due to negative 'financials.credit_history_months'.
 -> Deleted 1 records due to negative 'financials.savings_balance'.
 -> Deleted 11 records due to badly formatted emails.

Total invalid records successfully purged from database: 14
```

Data Curation Code

3.8 Standardizing annual_income to float

```python
# Fetch all documents that have an annual_income field
docs_with_income = collection.find({"financials.annual_income": {"$exists": True}})
bulk_operations = []

for doc in docs_with_income:
    doc_mongo_id = doc['_id']
    income = doc['financials']['annual_income']

    new_income_value = None

    # 1. Handle Strings
    if isinstance(income, str):
        try:
            # Strip out any commas, dollar signs, or spaces, then cast to float
            clean_string = re.sub(r'[^\d.]', '', income)
            new_income_value = float(clean_string)
        except ValueError:
            # If the string is completely unparseable (e.g. "Unknown"), we can set it to a default or None
            new_income_value = 0.0

    # 2. Handle Integers (and natively recast existing floats just to be safe)
    elif isinstance(income, (int, float)):
        new_income_value = float(income)

    # If we successfully parsed a new value, and it strictly needs updating
    # (Checking type ensures we don't unnecessarily update existing floats)
    if new_income_value is not None and type(income) is not float:
        bulk_operations.append(
            pymongo.UpdateOne(
                {"_id": doc_mongo_id},
                {"$set": {"financials.annual_income": new_income_value}}
            )
        )

# Execute the type casting in a single batch
if bulk_operations:
    result = collection.bulk_write(bulk_operations)
    print(f" -> Successfully converted 'annual_income' to float in {result.modified_count} documents.")
else:
    print(" -> All 'annual_income' fields are already correctly formatted as floats.")
```

```
 -> Successfully converted 'annual_income' to float in 485 documents.
```

3.9 Standardizing Gender Encoding

We will map "M" and "F" values to "Male" and "Female" respectively. The blank value "" will be mapped to "Unknown".

```python
print("\n--- STANDARDIZING CATEGORICAL DATA: GENDER ---")

# 1. Fix 'M' -> 'Male'
result_m = collection.update_many(
    {"applicant_info.gender": "M"},
    {"$set": {"applicant_info.gender": "Male"}}
)
print(f" -> Converted 'M' to 'Male' in {result_m.modified_count} documents.")

# 2. Fix 'F' -> 'Female'
result_f = collection.update_many(
    {"applicant_info.gender": "F"},
    {"$set": {"applicant_info.gender": "Female"}}
)
print(f" -> Converted 'F' to 'Female' in {result_f.modified_count} documents.")

# 3. Fix '' (empty string) -> 'Unknown'
result_empty = collection.update_many(
    {"applicant_info.gender": ""},
    {"$set": {"applicant_info.gender": "Unknown"}}
)
print(f" -> Converted empty strings '' to 'Unknown' in {result_empty.modified_count} documents.")

# 4. Optional: Verify the final clean state
pipeline = [
    {"$match": {"applicant_info.gender": {"$exists": True}}},
    {"$group": {"_id": "$applicant_info.gender", "count": {"$sum": 1}}}
]
final_genders = list(collection.aggregate(pipeline))
clean_breakdown = {g["_id"]: g["count"] for g in final_genders}

print(f"\nFinal cleaned gender breakdown: {clean_breakdown}")
```

```
--- STANDARDIZING CATEGORICAL DATA: GENDER ---
 -> Converted 'M' to 'Male' in 53 documents.
 -> Converted 'F' to 'Female' in 58 documents.
 -> Converted empty strings '' to 'Unknown' in 0 documents.

Final cleaned gender breakdown: {'Female': 244, 'Male': 242}
```

- All *"annual_income"* values were converted to float type
- Standardized *"gender"* values into "Female" and "Male"

Data Curation Code

```python
# Regex pattern for the strict standard we want: exactly YYYY-MM-DD
dob_standard_pattern = r"^\d{4}-\d{2}-\d{2}$"
```

```
--- STANDARDIZING DATE FORMATS: DATE OF BIRTH ---
 -> Successfully standardized 'date_of_birth' in 157 documents.
```

- Standardized *"date_of_birth"* values using a regular expression
- Mapped *"annual_salary"* to *"annual_income"*
- Dropped the *"notes"* field from any documents left that still contained it
- Imputed placeholder values for several fields with missing values
- Deleted row containing missing *"date_of_birth"* value

```
-> Injected placeholder '000-00-0000' for 'applicant_info.ssn' in 1 documents.
-> Injected placeholder '0.0.0.0' for 'applicant_info.ip_address' in 1 documents.
-> Injected placeholder 'Unknown' for 'applicant_info.gender' in 1 documents.
-> Injected placeholder '1900-01-01' for 'applicant_info.date_of_birth' in 1 documents.
-> Injected placeholder '00000' for 'applicant_info.zip_code' in 1 documents.
-> Injected placeholder 'Not Applicable' for 'decision.rejection_reason' in 282 documents.
-> Injected placeholder '0.0' for 'decision.interest_rate' in 205 documents.
-> Injected placeholder '0.0' for 'decision.approved_amount' in 205 documents.
-> Injected placeholder 'Not Specified' for 'loan_purpose' in 438 documents.
-> Injected placeholder '1970-01-01T00:00:00Z' for 'processing_timestamp' in 428 documents.
```

```python
result = collection.delete_many({"applicant_info.date_of_birth": "1900-01-01"})

print(f"Deleted {result.deleted_count} invalid applications missing a true Date of Birth.")
```

```
Deleted 1 invalid applications missing a true Date of Birth.
```

3.3. Map annual_salary to annual_income and drop notes

```python
print("\n--- STANDARDIZING SCHEMA: SALARY & NOTES ---")

# 1. Drop the 'notes' field globally
# We use update_many to instantly wipe this field from any document that has it.
notes_update_result = collection.update_many(
    {"notes": {"$exists": True}},
    {"$unset": {"notes": ""}}  # $unset permanently removes the key
)
print(f" -> Dropped 'notes' field from {notes_update_result.modified_count} documents.")


# 2. Map 'annual_salary' to 'annual_income' and drop 'annual_salary'
# We fetch only the documents that contain the rogue 'annual_salary' key
salary_docs = collection.find({"financials.annual_salary": {"$exists": True}})
bulk_operations = []

for doc in salary_docs:
    # Extract the value from the wrong key
    salary_value = doc['financials']['annual_salary']

    # Create an update operation that sets the new key AND unsets the old key simultaneously
    bulk_operations.append(
        pymongo.UpdateOne(
            {"_id": doc['_id']},
            {
                "$set": {"financials.annual_income": salary_value},
                "$unset": {"financials.annual_salary": ""}
            }
        )
    )

# Execute the mapping operations in a single batch
if bulk_operations:
    salary_result = collection.bulk_write(bulk_operations)
    print(f" -> Mapped 'annual_salary' to 'annual_income' in {salary_result.modified_count} documents.")
else:
    print(" -> No 'annual_salary' fields found to map.")
```

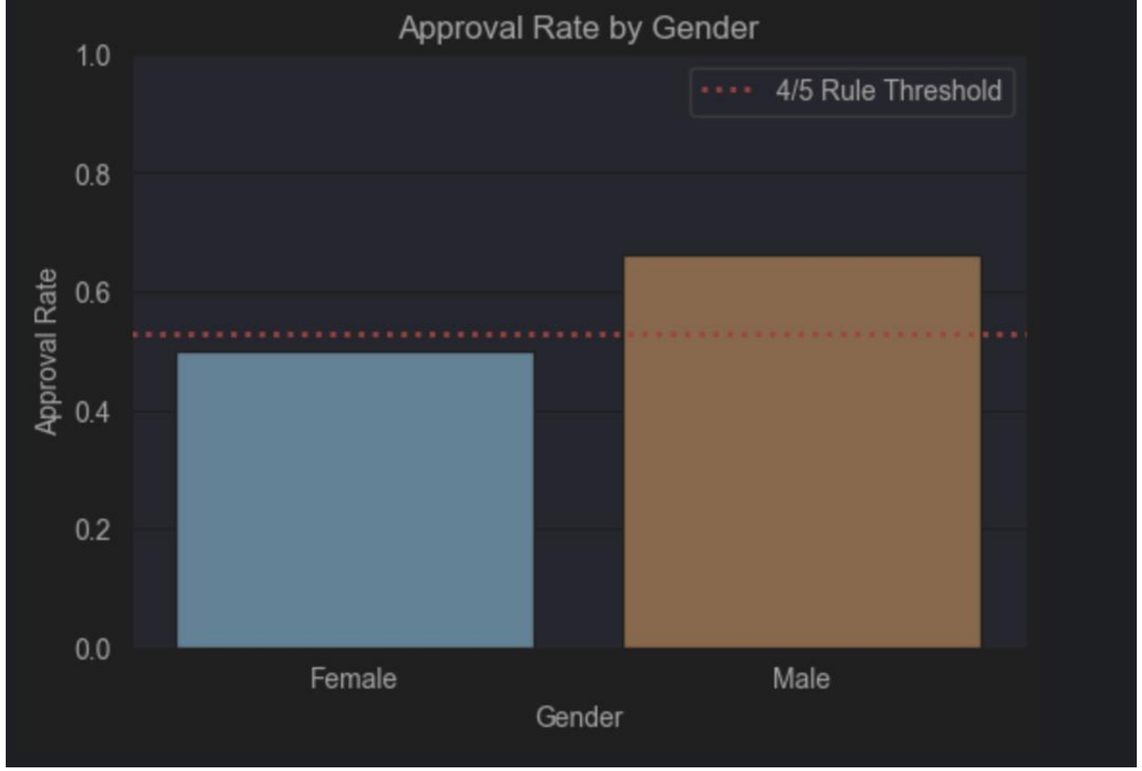
```
--- STANDARDIZING SCHEMA: SALARY & NOTES ---
 -> Dropped 'notes' field from 1 documents.
 -> Mapped 'annual_salary' to 'annual_income' in 5 documents.
```

Gender Bias Result

- Female approval rate is 50.0% vs 66.1% for males.
- Disparate Impact = 0.756 (< 0.80) -> fairness alert triggered.
- Statistical test confirms this is not random (chi2=11.21, p=0.000813).
- Fairlearn parity gap (DPD=0.161) supports meaningful outcome disparity.

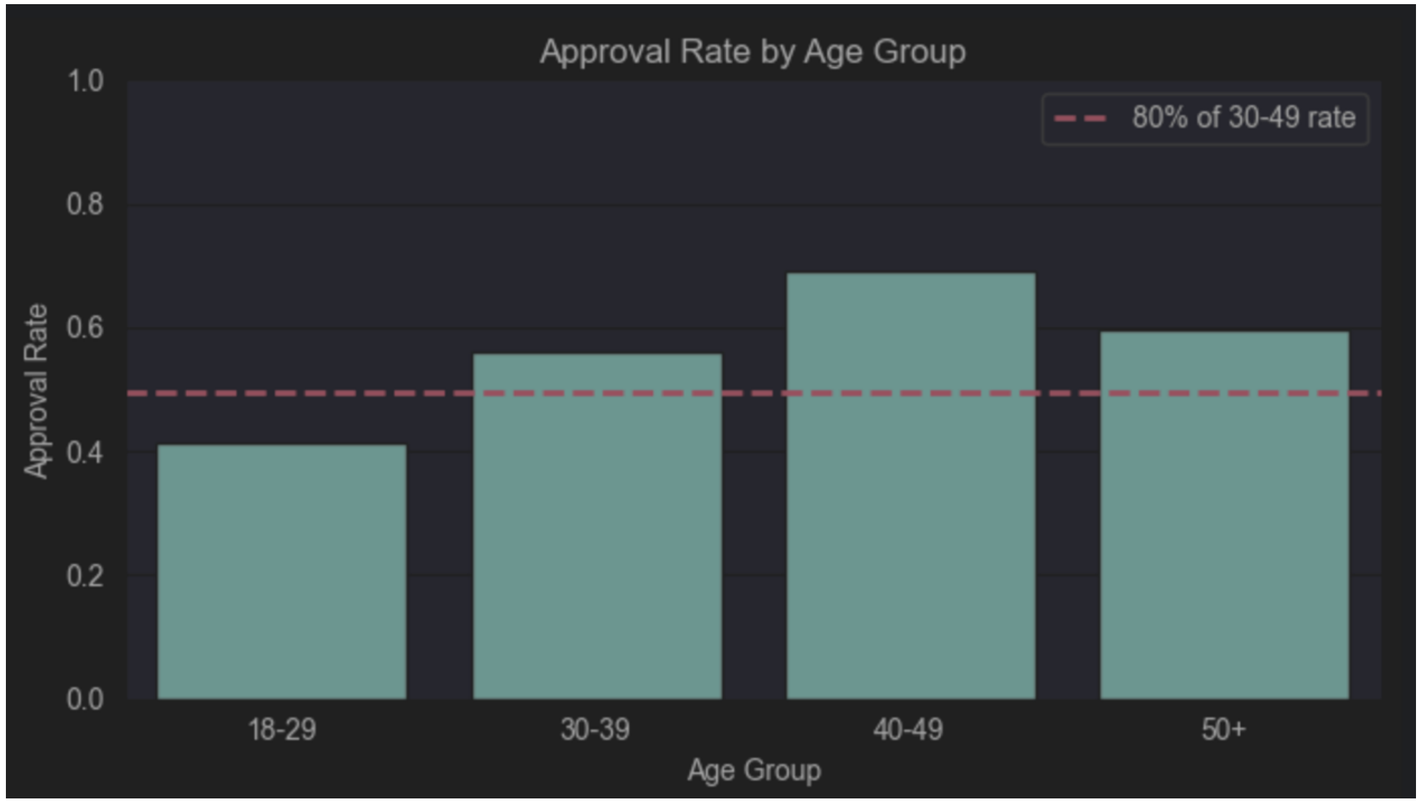
2 rows 2 rows × 1 cols

applicant_info.gender	approval_rate
Female	0.500000
Male	0.661157



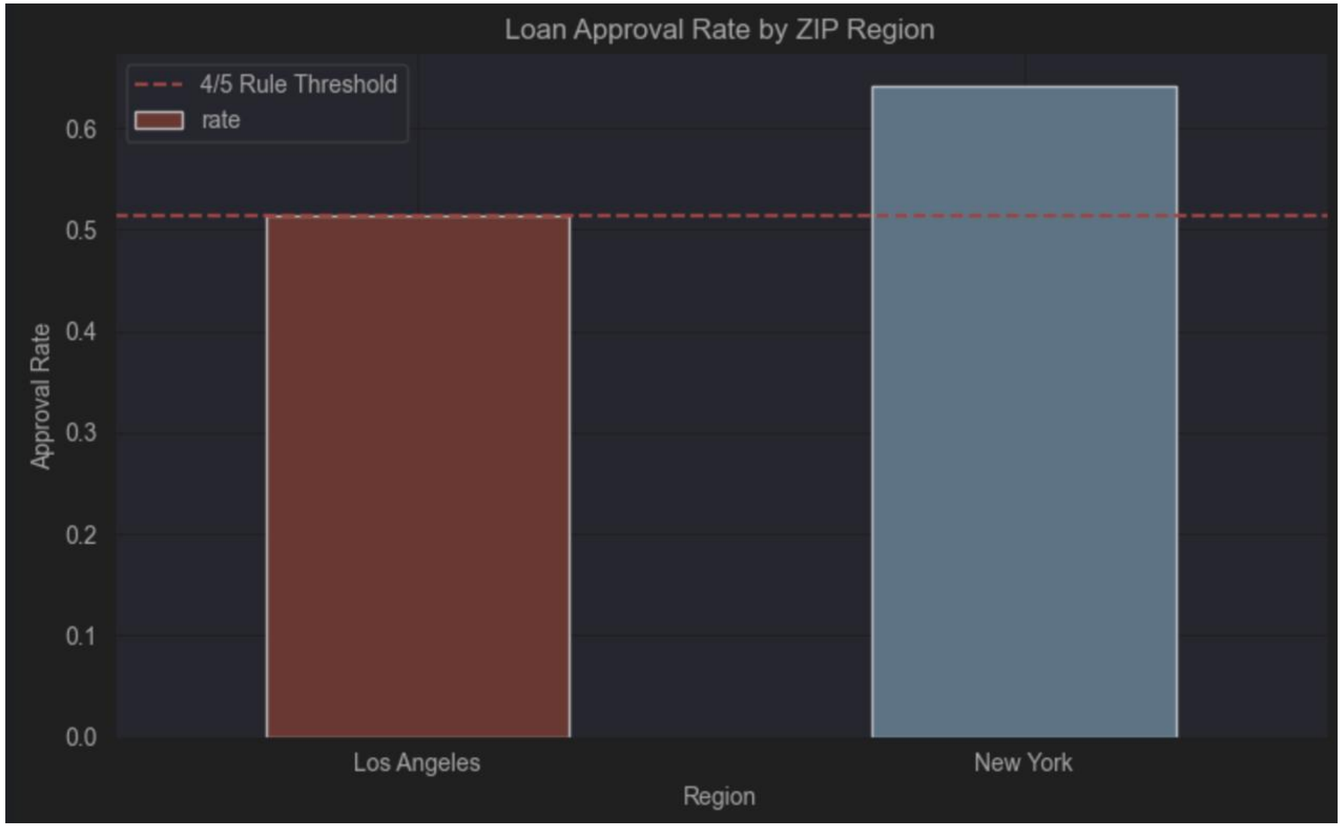
Age-group Approval Comparison

- Younger applicants (<30) are materially disadvantaged against the prime-age reference (30–49).
- Age effect is not uniform: the strongest gender gap appears in 25–34.
- Intersection analysis is statistically significant (chi2=31.38, p=0.000255).
- Interpretation: risk is amplified for specific subgroup combinations, not only single attributes.



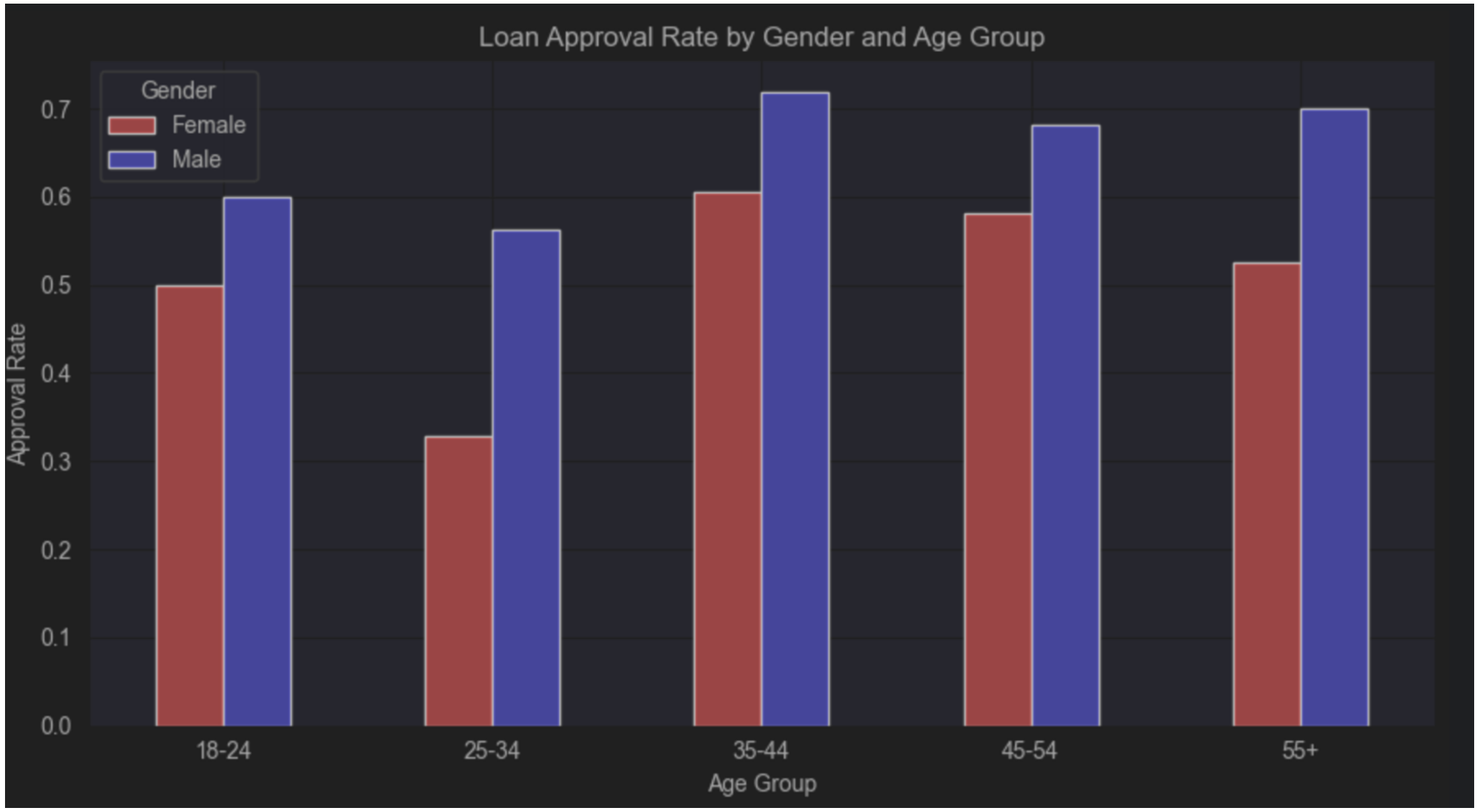
Proxy Variables Analysis

- Los Angeles approvals are lower than New York, with DI at the fairness threshold (0.7995).
- Region is significantly associated with outcomes (p=0.0064).
- Region is strongly associated with gender composition (p<0.001) -> proxy-discrimination risk.



Intersectional Effects (Gender × Age)

- Disparities are stronger when gender and age are analyzed jointly, not separately.
- Largest subgroup gap appears in age 25–34, with a 23.4 percentage-point male-female approval difference.
- Female-to-male DI varies by age group, indicating unequal impact concentration.
- Interpretation: fairness risk is intersectional; controls should monitor subgroup combinations, not only single protected attributes.



Privacy & Governance

Identifying PII

```
Direct Identifiers
[!] FOUND: 'applicant_info.full_name' is exposed in 495 records.
Evidence (Doc _id: 69ae1c9f2be98188be9fedc1): Jerry Smith
[!] FOUND: 'applicant_info.email' is exposed in 495 records.
Evidence (Doc _id: 69ae1c9f2be98188be9fedc1): jerry.smith17@hotmail.com
[!] FOUND: 'applicant_info.ssn' is exposed in 495 records.
Evidence (Doc _id: 69ae1c9f2be98188be9fedc1): 596-64-4340
```

```
Online Identifiers
[!] FOUND: 'applicant_info.ip_address' is exposed in 495 records.
Evidence (Doc _id: 69ae1c9f2be98188be9fedc1): 192.168.48.155
```

```
Quasi-Identifiers / Demographic
[!] FOUND: 'applicant_info.date_of_birth' is exposed in 495 records.
Evidence (Doc _id: 69ae1c9f2be98188be9fedc1): 2001-03-09
[!] FOUND: 'applicant_info.zip_code' is exposed in 495 records.
Evidence (Doc _id: 69ae1c9f2be98188be9fedc1): 10036
[!] FOUND: 'applicant_info.gender' is exposed in 495 records.
Evidence (Doc _id: 69ae1c9f2be98188be9fedc1): Male
```

```
--- Category: Financial & Behavioral Data ---
[!] EXPOSED: 'financials.annual_income' found in 495 records.
    Evidence (Doc ID: 69aef01c8bee53eb880ff2e9): 73000.0
[!] EXPOSED: 'spending_behavior.category' found in 495 records.
    Evidence (Doc ID: 69aef01c8bee53eb880ff2e9): ['Shopping', 'Rent', 'Alcohol']
```

Directly map to an individual

Direct Identifiers found in the datase:

- Full name
- Email
- SSN

These are ot unique alone but can identify someone when combined with other data

Quasi-Identifiers found in the datase:

- Date of birth
- Zip code
- Gender

Privacy & Governance

GDPR

Principle	GDPR Article	Technical Implementation in Code	Regulatory Goal
Security of Processing	**Article 32**	SHA-256 Pseudonymization	Protects PII (Email/SSN) using irreversible cryptographic hashing.
Lawfulness & Consent	**Article 7**	compliance.consent_status	Provides verifiable evidence of explicit user opt-in for data processing.
Storage Limitation	**Article 5.1.e**	retention_policy.expiry_date	Enforces a mandatory 5-year maximum lifespan for data storage.
Data Minimization	**Article 5.1.c**	drop_fields(['ip_address'])	Ensures only relevant data is processed, removing unnecessary identifiers.
National IDs	**Article 87**	hash_ssn()	Provides specific protection for government-issued identification numbers.

```
[EVIDENCE] Transformation for Record ID: app_200
Algorithm used: SHA-256 (Cryptographic Hashing)
_____________________________________________________________________
FIELD: Email

  AFTER : 116648a7761525746032d0ab323ceb01f50d11f793516437a5cf80804fc6fb9d
```

```
# Retention Policy
"retention_policy": {
    "expiry_date": expiry_date,
    "action": "permanent_deletion"
}
```

```
# Consent Mechanism (Art. 7 GDPR)
"compliance": {
    "consent_status": "explicit_opt_in",
    "consent_date": datetime.now().isoformat(),
    "legal_basis": "Art. 6(1)(a)"
},
```

Privacy & Governance

AI Act

Requirement	EU AI Act Article	Technical Implementation in Code	Purpose
Traceability	**Article 12**	audit_trail.model_version	Logs the specific AI version to ensure transparency in credit decisions.
Human Oversight	**Article 14**	human_oversight_required: True	Ensures high-risk rejections are flagged for manual review by an officer.
Data Quality	**Article 10**	data_quality_check: "Verified"	Validates that malformed data was audited and corrected.
Risk Classification	**Annex III**	risk_level: "High-Risk"	Formally categorizes the system under credit scoring regulatory rules.

Thank You

Any questions?